CSCI 3141
Foundations of Data Science in R *(and Python)*

DALHOUSIE UNIVERSITY

Reproducible Reporting

R Markdown and Python notebooks

Prof. Gabriel Spadon, PhD

Faculty of Computer Science

spadon@dal.ca

1 From an analysis to a report

What it is for A report tells a reader the question, the observations used, the preparation choices, the evidence and the limits of the conclusion. Reporting communicates the results of importing, summarizing, cleaning, transforming and visualizing, as Figure 1 shows.

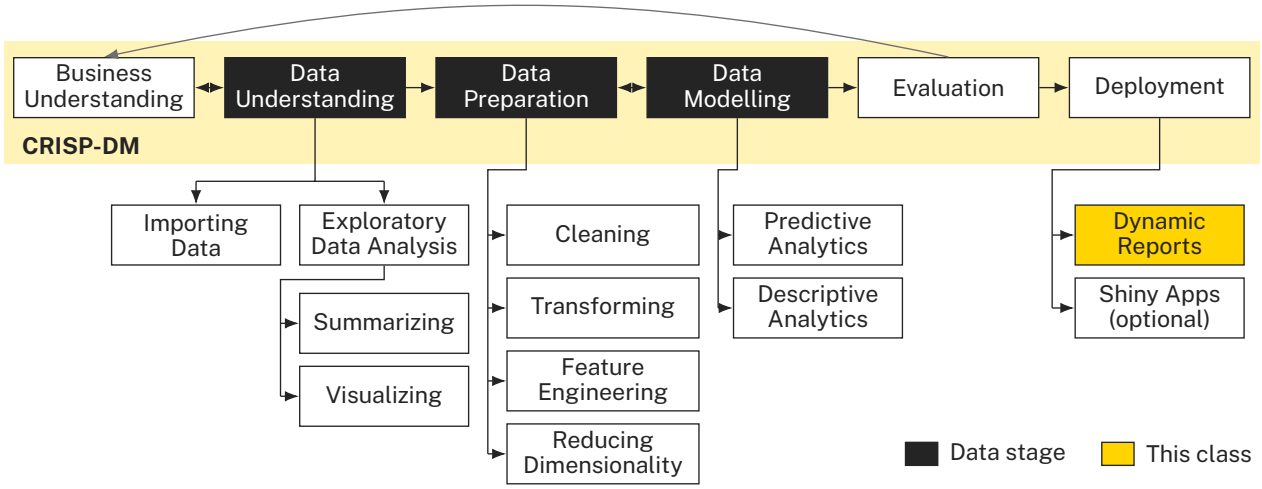


Figure 1. Dynamic reports communicate work from earlier stages. An interactive Shiny application is an optional sibling, not a required step after a report.

The problem with copied output Suppose a data correction changes the number of eligible records. You rerun the analysis and paste the new table, but a sentence still names the old count (Figure 2). A computed sentence prevents that mismatch once the report is rebuilt. It does not check the choice of denominator, and it does not update an interpretation written as ordinary text. Table 1 pairs each risk with the check that catches it.

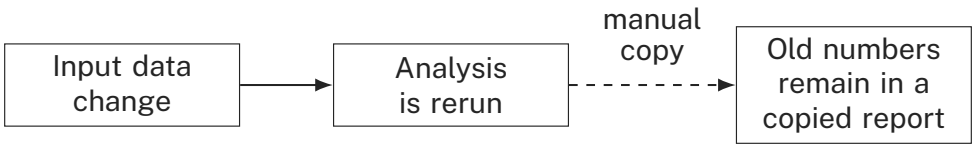


Figure 2. A manual copy can leave a report with old results after its analysis changes. Rebuilding linked outputs reduces that risk; checking the rendered report remains necessary.

Table 1. Risks and corresponding checks. No single source file replaces all of these checks.

Risk	Check in a dynamic workflow
Copied results become stale	Recompute tables, plots and numerical prose
Hidden session objects	Execute from a fresh process or kernel
Input file changes silently	Record input version or checksum
A cached result survives	Disable cache for the final small report
A chart changes meaning	Inspect labels, units, scale and caption
A reader cannot reproduce it	Retain source, data and environment record

Definition A *dynamic document* keeps explanatory text and executable code in one source file, so tables, figures and numerical sentences are rebuilt together (Figure 3). The source still needs its input data and its software. A PDF is a fixed rendering of one run, not the executable analysis.

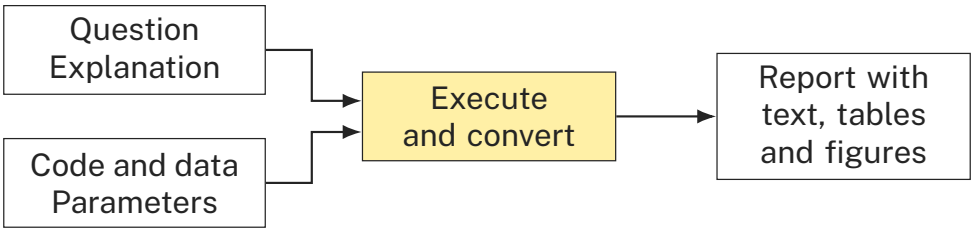


Figure 3. The ingredients of a dynamic document. This is a schematic workflow, not a screenshot of a particular editor.

2 Execution and conversion

What it is for Producing a PDF takes several separate programs. Knowing which one failed tells you where to look.

The R Markdown route Markdown marks document structure with plain text conventions for headings, paragraphs, emphasis, links and lists. An R Markdown file adds a YAML header that names the output and its settings, fenced code chunks that contain R, and inline R expressions that put computed values into sentences. The `rmarkdown` package runs three stages in order (Figure 4). knitr executes the enabled R chunks, Pandoc [4] converts the resulting Markdown, and a TeX engine (XeLaTeX here) compiles the PDF.

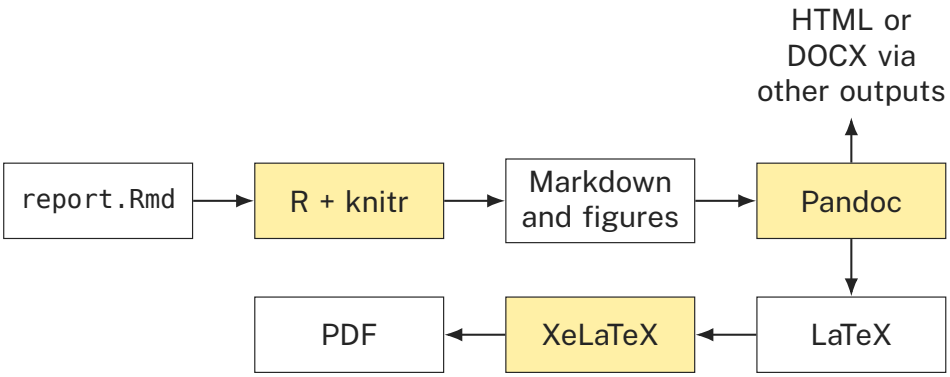


Figure 4. The R Markdown PDF route, with execution, document conversion and TeX compilation shown separately. Other output choices can produce HTML or DOCX. After Xie, Allaire and Grolemund [9].

The notebook route A Jupyter notebook is a JSON text file with an `.ipynb` extension [3]. It stores Markdown cells, code cells, metadata and possibly saved outputs. A fresh kernel executes the code cells in document order. nbconvert coordinates execution and export, Pandoc converts the Markdown, and XeLaTeX produces the PDF (Figure 5). Saved outputs and metadata make an `.ipynb` diff noisier than an `.Rmd` diff, but the file is still text and belongs in version control.

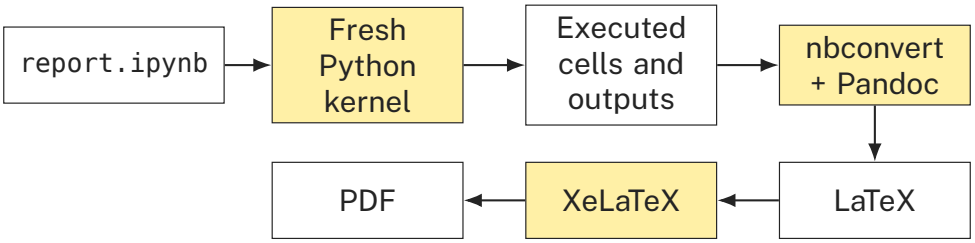


Figure 5. The notebook PDF route. Adding `--execute` is essential when the output must reflect a fresh run. Exporting saved outputs alone does not recompute them.

Locating a failure A failing code chunk or cell is an execution error, so check the named line and its inputs. A conversion failure is in the Pandoc log. A missing LaTeX package or invalid LaTeX is a TeX error. Locating the stage does not certify that earlier results are analytically correct.

3 Prepare one working environment

What it is for A report renders only where its packages, Pandoc and TeX are installed. Set this up once, before writing any report.

Folder rule Keep a report and its inputs in one known folder and run every command from that folder, so relative input paths resolve. For the assessed lab, save `report.Rmd` or `report.ipynb` in the `labs/lab-04` folder of your own course repository. Build each class example in its own practice folder so it does not overwrite your lab work. All code and the CSV appear in this handout.

3.1 R Markdown

Use R with RStudio Desktop or another editor. Install the packages once in the R console, not inside each report.

```
install.packages(c("rmarkdown", "knitr", "ggplot2", "dplyr"))
```

PDF output also needs Pandoc and a TeX distribution containing XeLaTeX. RStudio bundles Pandoc for its own buttons, but a terminal must be able to find Pandoc too. Check each tool before diagnosing an installation problem.

```
Rscript --version
pandoc --version
xelatex --version
```

If TeX is absent, the TinyTeX installation guide describes a lightweight R-compatible distribution. An existing working TeX installation is also suitable. HTML needs no TeX and is useful for checking, but the Lab 04 output is PDF.

3.2 Python notebooks

Use JupyterLab locally or a supported notebook environment. With your Python environment active, install the dependencies once and register a kernel. The kernel's environment must contain these packages.

```
python -m pip install jupyterlab nbconvert ipykernel pandas numpy matplotlib scikit-learn jinja2
python -m ipykernel install --user --name csci3141 --display-name "CSCI 3141"
jupyter lab
```

Create a Python notebook, choose the `CSCI 3141` kernel, and save it as `report.ipynb`. The PDF exporter also needs Pandoc and XeLaTeX. On the command line, adding this option to `jupyter nbconvert` selects the kernel explicitly.

```
--ExecutePreprocessor.kernel_name=csci3141
```

For another prepared kernel, substitute its registered name and check its package versions.

3.3 Local and cloud choices

A local RStudio or Jupyter project keeps files on your computer and gives direct control over installed tools. Posit Cloud runs browser-based projects whose files can be downloaded; keep the report, input and output together and check the sharing settings. Google Colab saves the notebook to Drive or as a download, but its runtime and any files stored only on that runtime can disappear, so save inputs and outputs deliberately. Kaggle notebooks are another option listed in the syllabus; check the language, installed packages, file persistence and export before starting.

If a cloud environment cannot produce the PDF, download the source and inputs and render them locally, early enough to inspect the result. Do not make coursework public to transfer or export it. Use the course's private submission workflow until the deadline and instructor release. Current guidance is in Posit Cloud projects and the Colab FAQ.

4 A complete Iris report in R Markdown

What it is for This section builds one full report, block by block, so each part of the source has a visible effect in the PDF.

The data Iris has 150 flowers, 50 of each of three species, with measurements in centimetres [2, 6]. It ships with R and with scikit-learn, and the two copies agree, so there is no CSV to download.

Create `report.Rmd` as a plain-text file and append the blocks below in order. The triple backticks, the chunk headers and the paragraphs inside the listings are all part of the file. A `##` line is a second-level Markdown heading.

4.1 Document settings and setup

The YAML header sits between two lines of `---`. Indent nested settings with spaces, two per level. The first listed format, `pdf_document`, is the default render; `html_document` is a second format a render command can select. The blank date omits the date from the title. Put your own name in the author field.

```
---
title: "Iris sepal lengths"
author: "Course example"
date: ""
output:
  pdf_document:
    latex_engine: xelatex
    fig_caption: true
  html_document: default
---
```

The setup chunk loads ggplot2 and dplyr, sets shared options and assigns the input. `include=FALSE` runs the chunk but hides its code and output; it does not skip execution. `cache=FALSE` keeps this small analysis fresh. Nothing here is random, so no seed is needed.

````
```{r setup, include=FALSE}
library(ggplot2)
library(dplyr)
knitr::opts_chunk$set(
  echo = FALSE, cache = FALSE, error = FALSE,
  warning = TRUE, message = FALSE,
  fig.width = 6, fig.height = 3, out.width = "90%"
)
dat <- iris
```
````

4.2 Computed prose

An inline expression starts with a backtick and `r` and ends with a backtick. `sprintf` fixes the number of displayed decimals.

```
## Question and data

How do sepal lengths vary within and between the three species in this
sample? Iris contains 150 flowers, 50 of each species, with measurements
in centimetres.

The mean sepal length is `r sprintf("%.2f", mean(dat$Sepal.Length))` cm,
with sample standard deviation `r sprintf("%.3f", sd(dat$Sepal.Length))` cm.

## A compact summary
```

Rendered, the sentence reads a mean of 5.84 cm and a sample standard deviation of 0.828 cm. Both are recomputed on every render. R's `sd` divides by $n - 1$.

4.3 A compact table

For each species, `group_by` and `summarise` compute the count and two means. `.groups='drop'` returns an ordinary table without grouping, and `row.names=FALSE` stops `kable` adding a redundant row-label column.

````
```{r species-summary}
summary_table <- dat |>
  group_by(Species) |>
  summarise(
    n = n(), Sepal_cm = mean(Sepal.Length),
    Petal_cm = mean(Petal.Length), .groups = "drop"
  )
knitr::kable(summary_table, row.names = FALSE, digits = 3)
```
````

```
The table reports means in centimetres. Each species has the same sample
size, but a mean alone does not describe the within-species spread.

## Two views of the variation
```

Table 2 shows the same values with readable headers. Every species has 50 flowers. Means alone hide the spread, so the report also needs plots.

Table 2. The calculated Iris summary, with all measurement means in centimetres.

Species	n	Sepal mean (cm)	Petal mean (cm)
setosa	50	5.006	1.462
versicolor	50	5.936	4.260
virginica	50	6.588	5.552

4.4 Two complementary figures

The histogram shows the pooled distribution; grouped box plots compare the species. The histogram uses an explicit bin grid, 0.4 cm wide from 4.25 to 8.25 cm. No measurement lies on an internal edge, so the bin rule never has to break a tie. The ten counts are 9, 23, 20, 28, 19, 23, 16, 6, 5 and 1, which sum to 150 (Figure 6).

````
```{r histogram, fig.cap="Sepal length distribution."}
edges <- seq(4.25, 8.25, length.out = 11)
ggplot(dat, aes(Sepal.Length)) +
  geom_histogram(
    breaks = edges, closed = "left",
    fill = "#6E6E6E", colour = "white"
  ) +
  labs(x = "Sepal length (cm)", y = "Count") +
  theme_minimal(base_size = 11)
```
````

```
Figure 1 uses 0.4 cm bins from 4.25 to 8.25 cm. The pooled histogram
combines species. Its bin edges avoid the recorded
0.1 cm measurement grid, so no observation lies on an internal edge.
```

````
```{r boxplot, fig.cap="Sepal lengths by species."}
ggplot(dat, aes(Species, Sepal.Length)) +
  geom_boxplot(fill = "#F2F2F2", width = 0.55) +
  labs(x = "Species", y = "Sepal length (cm)") +
  theme_minimal(base_size = 11)
```
````

```
Figure 2 uses linear quartiles and 1.5-IQR whiskers.
Setosa has the lowest sample median, while the species' ranges overlap.
The box plots reveal variation hidden by the table. They do not establish
that species alone causes a difference in sepal length.

## Reproduction record

Data are the built-in R `iris` dataset. The histogram uses the explicit
edges above; ggplot2 uses linearly interpolated quartiles for the boxes.
No random operation or cached analysis is used.
```

Reading the box plot The box edges are Q_1 and Q_3, the central line is the median, and the whiskers reach the most extreme observations within $1.5\,\mathrm{IQR}$ of the box. For virginica, linear interpolation gives $Q_1 = 6.225$ cm, and the lower fence is 5.2125 cm. The smallest value, 4.9 cm, lies beyond it and is drawn as a point; the next, 5.6 cm, ends the whisker (Figure 7). A flagged point is not automatically an error. Base R's `boxplot` uses Tukey hinges [8] instead, which can differ slightly; its virginica lower hinge is 6.2 cm.

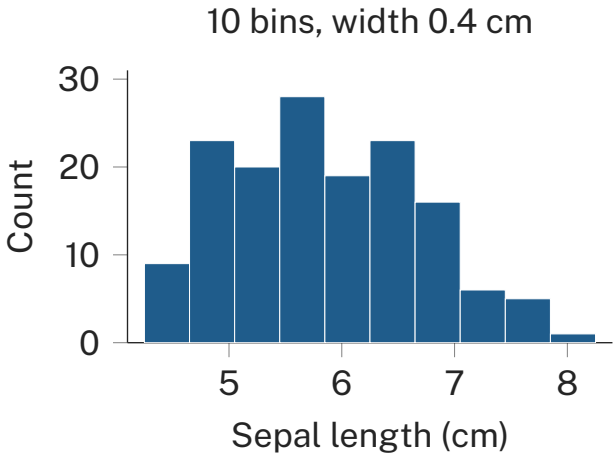


Figure 6. The pooled sepal-length histogram seen before. The 0.4 cm bins begin at 4.25 cm; the vertical axis counts flowers.

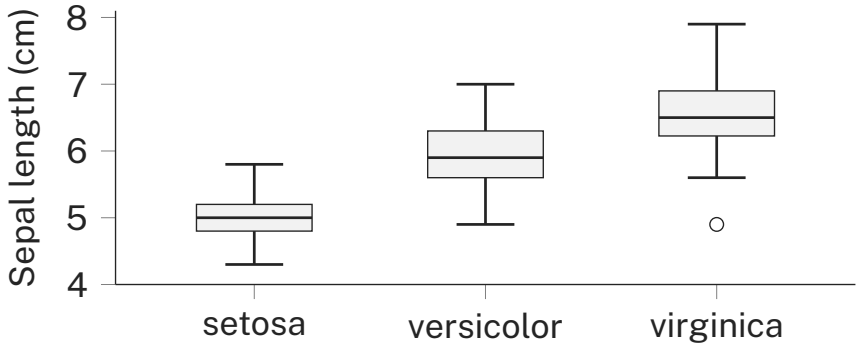


Figure 7. Sepal lengths by species seen before, using linearly interpolated quartiles. The virginica lower whisker is 5.6 cm, and the separate 4.9 cm observation lies beyond the lower fence.

4.5 Record the environment

Finish the file with the version output. Versions help explain later changes in default behaviour. They do not replace keeping the data or recording the analysis choices.

````
```{r versions}
cat("R", as.character(getRversion()), "\n")
for (pkg in c("ggplot2", "dplyr", "knitr", "rmarkdown")) {
  cat(pkg, as.character(packageVersion(pkg)), "\n")
}
```
````

5 The same Iris report in a Python notebook

What it is for The same report, with the same numbers, one table and two figures, built from notebook cells.

Create the cells below in order in `report.ipynb`. The editor stores the cell structure, so never type JSON. Put Markdown text in a Markdown cell and code in a Code cell, as labelled.

5.1 Question, imports and computed prose

Markdown cell

```
# Iris sepal lengths

How do sepal lengths vary within and between the three species in this
sample? Iris contains 150 flowers, 50 of each species, with measurements
in centimetres.
```

Code cell

```
import platform
from importlib.metadata import version

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from IPython.display import Markdown, display
from sklearn.datasets import load_iris

iris = load_iris()
dat = pd.DataFrame(
    iris.data,
    columns=["Sepal.Length", "Sepal.Width", "Petal.Length", "Petal.Width"],
)
dat["Species"] = iris.target_names[iris.target]
species = list(iris.target_names)
```

The setup builds a table from scikit-learn's bundled Iris data and gives the columns R's names, so both languages refer to `Sepal.Length`.

Code cell

```
mean_sepal = dat["Sepal.Length"].mean()
sd_sepal = dat["Sepal.Length"].std(ddof=1)
display(
    Markdown(
        f"The mean sepal length is {mean_sepal:.2f} cm, "
        f"with sample standard deviation {sd_sepal:.3f} cm."
    )
)
```

A Markdown cell does not evaluate Python. Instead, `display(Markdown(...))` emits formatted text from a code cell, and the f-string inserts the values with the same precision as the R sentence. `ddof=1` gives the $n-1$ sample standard deviation that R uses. The exported PDF shows the computed output.

5.2 The table

Markdown cell

```
## A compact summary
```

Code cell

```
summary_table = (
    dat.groupby("Species", sort=True)
    .agg(
        n=("Sepal.Length", "size"),
        Sepal_cm=("Sepal.Length", "mean"),
        Petal_cm=("Petal.Length", "mean"),
    )
    .reset_index()
)
display(
    {
        "text/html": summary_table.to_html(
            index=False, float_format="%.3f", escape=True
        ),
        "text/latex": summary_table.to_latex(
            index=False, float_format="%.3f", escape=True
        ),
    },
    raw=True,
)
```

Markdown cell

```
The table reports means in centimetres. Each species has the same sample
size, but a mean alone does not describe the within-species spread.

## Two views of the variation
```

The grouped aggregation computes the count and two means, then restores Species as an ordinary column. The display dictionary carries two representations of the same table. A notebook or HTML export uses `text/html`; the PDF exporter uses `text/latex`. `escape=True` protects special characters in cell values. pandas writes both, so you write neither by hand.

5.3 The two plots and interpretation

Code cell

```
edges = np.linspace(4.25, 8.25, 11)
fig, ax = plt.subplots(figsize=(6, 3))
ax.hist(
    dat["Sepal.Length"],
    bins=edges,
    color="#6E6E6E",
    edgecolor="white",
)
ax.set(xlabel="Sepal length (cm)", ylabel="Count")
ax.set_title("Figure 1. Sepal length distribution", fontsize=11)
fig.tight_layout()
plt.show()
```

Markdown cell

```
The histogram uses 0.4 cm bins from 4.25 to 8.25 cm and combines species.
The edges avoid the recorded
0.1 cm measurement grid, so no observation lies on an internal edge.
```

Code cell

```
groups = [dat.loc[dat["Species"].eq(name), "Sepal.Length"] for name in species]
fig, ax = plt.subplots(figsize=(6, 3))
ax.boxplot(
    groups,
    tick_labels=species,
    widths=0.55,
    patch_artist=True,
    boxprops={"facecolor": "#F2F2F2"},
    medianprops={"color": "black"},
)
ax.set(xlabel="Species", ylabel="Sepal length (cm)")
ax.set_title("Figure 2. Sepal lengths by species", fontsize=11)
fig.tight_layout()
plt.show()
```

Markdown cell

```
The boxes use linear quartiles and 1.5-IQR whiskers.
Setosa has the lowest sample median, while the species'
ranges overlap. The boxes reveal variation hidden by the table. They
do not establish that species alone causes a sepal-length difference.

## Reproduction record
Data are the built-in scikit-learn Iris dataset, matching R's 150 rows.
The histogram uses the explicit edges above; matplotlib uses linearly
interpolated quartiles. No random operation or cached analysis is used.
```

Each plot gets its own axes and axis labels, `tight_layout()` fits the labels, and the figure is displayed. Matplotlib uses linear quartiles and $1.5\,\mathrm{IQR}$ whiskers here, so the plots use the same bins, quartiles and whiskers as Figures 6 and 7.

Code cell

```
print("Python", platform.python_version())
for package in [
    "pandas",
    "numpy",
    "matplotlib",
    "scikit-learn",
    "IPython",
    "nbconvert",
    "Jinja2",
]:
    print(package, version(package))
```

Differences to know

- R evaluates an inline expression inside the text. A notebook Markdown cell evaluates nothing, so a code cell emits the sentence with `display(Markdown(...))`.
- R's `sd` and pandas' `std` both divide by $n-1$; NumPy's `np.std` divides by n unless given `ddof=1`, so the notebook states `ddof=1` explicitly.
- R formats a data frame with `kable`; pandas supplies HTML and LaTeX representations and the exporter picks one.
- ggplot2 and matplotlib use linear quartiles here; base R `boxplot` uses Tukey hinges.
- R sets a caption with `fig.cap`; the notebook puts a numbered title inside the figure and its explanation in the next Markdown cell.

6 Render and inspect the PDF

What it is for Only a render from a fresh process shows that the source, not your interactive session, produces the result.

The commands Run the one for your language in the report folder. The R command starts a new R process. The notebook command starts a kernel and executes the cells in order before converting.

```
Rscript --vanilla -e "rmarkdown::render(
  'report.Rmd', output_format='pdf_document')"
```

```
jupyter nbconvert --execute --to pdf --no-input report.ipynb
```

Both write `report.pdf`. `--no-input` hides code in the PDF; `--execute` still runs it, and the notebook keeps its code. Omit `--no-input` during development when seeing code beside output helps. Never use `--allow-errors` for a final report.

For a quick HTML check, use these instead. HTML does not replace the assessed PDF.

```
Rscript --vanilla -e "rmarkdown::render(
  'report.Rmd', output_format='html_document')"
```

```
jupyter nbconvert --execute --to html --no-input --embed-images report.ipynb
```

Checking the result A successful command shows that this configured run finished. It does not show the analysis is correct. After every final render,

1. confirm the command finished without an execution error;
2. open the new PDF and inspect every page for missing plots, unreadable labels, broken tables, clipped code and unexpected warnings;
3. check the count and one calculated value against the input (150 flowers, mean 5.84 cm);
4. check every unit and that each interpretation still matches the results; and

5. keep the source, input identity and software versions with the PDF.

7 Control execution and presentation

What it is for Chunk options decide separately what code runs and what the reader sees. Figure settings decide how large a plot is drawn and shown.

7.1 R chunk options

Give every chunk a short, unique label such as `species-summary`, so logs and figure files are easy to identify. Hyphens are a convention; a period is not forbidden. Table 3 lists the common options and their defaults.

Table 3. Common R Markdown chunk defaults. Standalone knitr has a different default for `error`; this table describes R Markdown rendering.

Option	R Markdown default	Effect
`eval`	`TRUE`	Run the code
`echo`	`TRUE`	Show source code
`include`	`TRUE`	Include source and output
`results`	`'markup'`	Format text output
`message`	`TRUE`	Include messages
`warning`	`TRUE`	Include warnings
`error`	`FALSE`	Stop on a code error
`cache`	`FALSE`	Reuse cached results if enabled

Running versus showing `eval=FALSE` disables execution, so that chunk cannot supply an object needed later. `echo=FALSE` hides the source code. `include=FALSE` hides both source and output while still running the chunk. To see the difference, change the Iris histogram header to one of these and render each version.

````
```{r histogram, eval=FALSE, echo=TRUE}
```{r histogram, eval=TRUE, echo=FALSE}
````

Keep the existing code and closing backticks after the chosen header, use only one version at a time, and restore execution before the final PDF. The required analysis must run when the report renders.

Text, messages and errors `results='asis'` lets code emit Markdown structure; `results='hide'` omits text results. `message=FALSE` suppresses messages. `warning=FALSE` suppresses warnings everywhere, including the render log, so keep warnings on while checking the analysis. A warning can matter even when a PDF was produced. `error=FALSE` stops rendering at a code error.

7.2 Figure size, display width and captions

`fig.width` and `fig.height` set the R graphics device size in inches, or `fig.asp` sets the height-to-width ratio. `out.width`, such as `'90%'`, sets the display width in the document. In Python, `figsize=(6,3)` draws a six-by-three inch figure. Drawing large and shrinking afterwards shrinks the labels too, so check the size in the exported PDF.

`fig.cap` supplies an R caption, and the example's `fig_caption: true` numbers it in PDF output. Numbering and float placement depend on the output format. The notebook example puts a numbered title inside each figure, which stays with the plot on export, and explains units and plotting convention in the next Markdown cell. Keep explanations next to their figures, and renumber by hand if you add or remove a plot.

7.3 Formats and saved notebook state

Table 4 lists the core output choices in each language.

Table 4. Core output choices. The two languages have different native format names.

Output	R Markdown	Python notebook
PDF	`pdf_document`	`nbconvert --to pdf`
HTML	`html_document`	`nbconvert --to html`
Editable source	`.Rmd`	`.ipynb`
R notebook preview	`html_notebook`	Not a Python format

Saved output can come from an earlier run. An R `html_notebook` shows interactive chunk output and an HTML preview, and that preview can keep outputs from earlier execution. A Python notebook can keep outputs from cells run out of order, a pattern that large studies of public notebooks find common [5]. An output-format name does not start a fresh session. An `html_document` is a rendered report, and `rmarkdown::render` evaluates in the calling environment unless told otherwise. Rebuild with `Rscript --vanilla` or `--execute` before the final export.

Quarto [1] also renders R and Python; learn one complete route before adding another tool.

8 A parameterized report

What it is for A *parameter* names an input choice, such as a threshold, so one analytical recipe can be rerun with a different value and the report records which value it used (Figure 8). This section combines the report structure, chunk options and rendering commands of the earlier sections.

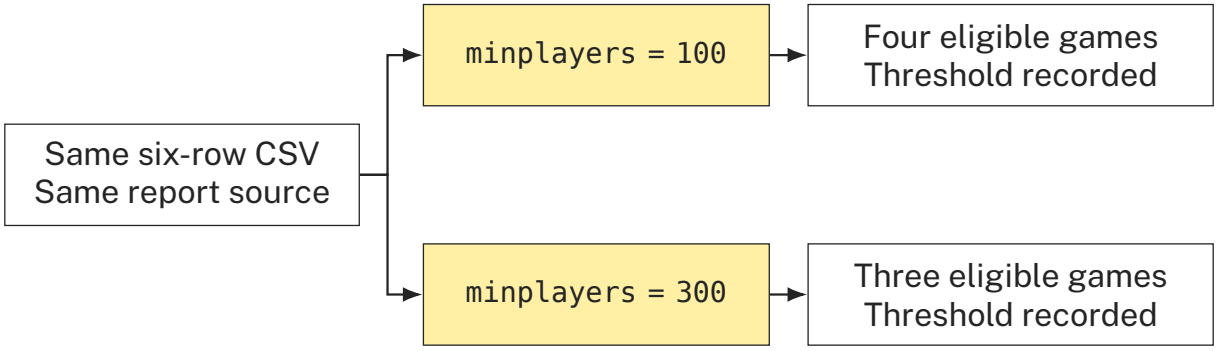


Figure 8. A schematic parameter comparison. Increasing the threshold to 300 retains three games and changes the reported coverage.

The data Six game-level records. Save these seven lines as `steam_achievements.csv` beside a new `report.Rmd` or `report.ipynb`, in a new practice folder. Keep the raw CSV unchanged and create filtered results in memory.

```
game,players,completed,achievements
Celeste,412,254,32
Hades,388,180,49
Stardew,301,112,40
Factorio,154,39,27
Spelunky,88,21,25
Tunic,64,12,26
```

One row is one game. `players` is the listed player count, `completed` counts the players who completed it, and `achievements` counts its available achievements.

The rule

$$\text{completion rate} = \frac{\text{completed players}}{\text{listed players}}$$

The denominator is players, never the achievement count. The rate is defined only when players is positive, and completed cannot exceed players. The source stops on a missing value, a nonpositive player count or more completions than players, rather than inventing replacements. Check by eye that the counts are whole numbers and that each game appears once.

8.1 R Markdown source

Append these blocks in order. The YAML defaults name a sibling input file and the threshold 100. A replacement `datafile` must have the same schema; an Iris file does not.

````
---
title: "Completion rates by game"
author: "Course example"
date: ""
output:
  pdf_document:
    latex_engine: xelatex
    fig_caption: true
  html_document: default
params:
  datafile: "steam_achievements.csv"
  minplayers: 100
---
````

````
```{r setup, include=FALSE}
library(ggplot2)
knitr::opts_chunk$set(
  echo = FALSE, cache = FALSE, error = FALSE,
  warning = TRUE, message = FALSE,
  fig.width = 6, fig.height = 3, out.width = "90%"
)
```
````

```
## Question and data

How do completion rates compare among games with at least the selected
number of players? A row represents one game; `completed` is the
number of its listed players who completed it. `achievements` counts
available achievements and is not the completion-rate denominator.
```

````
```{r prepare}
raw <- read.csv(params$datafile, stringsAsFactors = FALSE)
stopifnot(!anyNA(raw))
stopifnot(all(raw$players > 0))
stopifnot(all(raw$completed <= raw$players))
cleaned <- raw[raw$players >= params$minplayers, ]
cleaned$rate <- cleaned$completed / cleaned$players
cleaned <- cleaned[order(-cleaned$rate, cleaned$game), ]
```
````

````
## Coverage and comparison

The threshold is at least `r params$minplayers` players per game.
This retains `r nrow(cleaned)` of `r nrow(raw)` games.
````

````
```{r table}
shown <- cleaned[c("game", "players", "completed", "achievements", "rate")]
knitr::kable(shown, row.names = FALSE, digits = 3)
```
````

````
The completion rate divides `completed` by `players`. Counts and rates
describe different aspects of these records.
````

````
```{r rates, fig.cap="Completion rates for eligible games."}
if (nrow(cleaned) == 0) {
  cat("No games meet this threshold. There are no rates to plot.\n")
} else {
  cleaned$game <- factor(cleaned$game, levels = rev(cleaned$game))
  print(ggplot(cleaned, aes(rate, game)) +
    geom_col(fill = "#1F5C8B") +
    scale_x_continuous(limits = c(0, 1)) +
    labs(x = "Completed / players", y = "Game") +
    theme_minimal(base_size = 11))
}
```
````

````
```{r players, fig.cap="Player counts for eligible games."}
if (nrow(cleaned) > 0) {
  print(ggplot(cleaned, aes(players, game)) +
    geom_col(fill = "#6E6E6E") +
    labs(x = "Number of players", y = "Game") +
    theme_minimal(base_size = 11))
}
```
````

````
Filtering changes coverage, not the underlying per-game rate. When games
qualify, the player-count plot makes their denominators visible.

## Reproduction record
````

````
```{r versions}
cat("Input", params$datafile, "\n")
cat("Input MD5", unname(tools::md5sum(params$datafile)), "\n")
cat("R", as.character(getRversion()), "\n")
for (pkg in c("ggplot2", "knitr", "rmarkdown")) {
  cat(pkg, as.character(packageVersion(pkg)), "\n")
}
```
````

The assertions stop on invalid input. The filter leaves the original `raw` table intact. The table shows numerator and denominator, and the two plots separate the rate from the player count.

8.2 Python notebook cells

Create the cells in order. The parameter cell is ordinary Python assignment. To make a second report, edit `minplayers` and run from the beginning.

Markdown cell

```
# Completion rates by game

How do completion rates compare among games with at least the selected
number of players? Each row is one game. The completed column
counts listed players who completed it; achievements counts available
achievements and is not the completion-rate denominator.
```

Code cell

```
import hashlib
import platform
from importlib.metadata import version
from pathlib import Path

import matplotlib.pyplot as plt
import pandas as pd
from IPython.display import Markdown, display
```

Code cell, parameters

```
datafile = "steam_achievements.csv"
minplayers = 100
```

Code cell

```
raw = pd.read_csv(datafile)
assert raw.notna().all().all()
assert (raw["players"] > 0).all()
assert (raw["completed"] <= raw["players"]).all()
cleaned = raw.loc[raw["players"].ge(minplayers)].copy()
cleaned["rate"] = cleaned["completed"] / cleaned["players"]
cleaned = cleaned.sort_values(["rate", "game"], ascending=[False, True])
```

Code cell

```
display(
    Markdown(
        "## Coverage and comparison\n\n"
        f"The threshold is at least {minplayers} players per game. "
        f"This retains {len(cleaned)} of {len(raw)} games."
    )
)
```

Code cell

```
shown = cleaned[["game", "players", "completed", "achievements", "rate"]]
display(
    {
        "text/html": shown.to_html(
            index=False, float_format="%.3f", escape=True
        ),
        "text/latex": shown.to_latex(
            index=False, float_format="%.3f", escape=True
        ),
    },
    raw=True,
)
```

Markdown cell

```
The completion rate divides completed by players. Counts and rates
describe different aspects of these records.
```

Code cell

```
if cleaned.empty:
    display(
        Markdown(
            "No games meet this threshold. There are no rates to plot."
        )
    )
else:
    fig, ax = plt.subplots(figsize=(6, 3))
    ax.barh(cleaned["game"], cleaned["rate"], color="#1F5C8B")
    ax.invert_yaxis()
    ax.set(xlim=(0, 1), xlabel="Completed / players", ylabel="Game")
    ax.set_title("Figure 1. Completion rates", fontsize=11)
    fig.tight_layout()
    plt.show()
```

Markdown cell

```
Rates use a zero-to-one scale. No figure is drawn when no games are eligible.
```

Code cell

```
if not cleaned.empty:
    fig, ax = plt.subplots(figsize=(6, 3))
    ax.barh(cleaned["game"], cleaned["players"], color="#6E6E6E")
    ax.invert_yaxis()
    ax.set(xlabel="Number of players", ylabel="Game")
    ax.set_title("Figure 2. Player counts", fontsize=11)
    fig.tight_layout()
    plt.show()
```

Markdown cell

```
When games qualify, the player-count plot uses the same games and order
as the rate plot, making the denominators visible. Filtering changes
coverage, not the underlying per-game rate.

## Reproduction record
```

Code cell

```
print("Input", datafile)
print(
    "Input MD5",
    hashlib.md5(
        Path(datafile).read_bytes(), usedforsecurity=False
    ).hexdigest(),
)
print("Python", platform.python_version())
for package in [
    "pandas",
    "numpy",
    "matplotlib",
    "IPython",
    "nbconvert",
    "Jinja2",
]:
    print(package, version(package))
```

8.3 Worked example with thresholds 100, 300 and 1000

With the default threshold of 100 players, four of the six games remain; Spelunky (88) and Tunic (64) fall below it. Celeste's rate is $254/412 = 0.617$, and Table 5 gives the rest. Figures 9 and 10 keep the games in the same order, so each rate can be read beside the number of players it describes.

Table 5. Games with at least 100 players. Rate is completed divided by players.

Game	Players	Completed	Achievements	Rate
Celeste	412	254	32	0.617
Hades	388	180	49	0.464
Stardew	301	112	40	0.372
Factorio	154	39	27	0.253

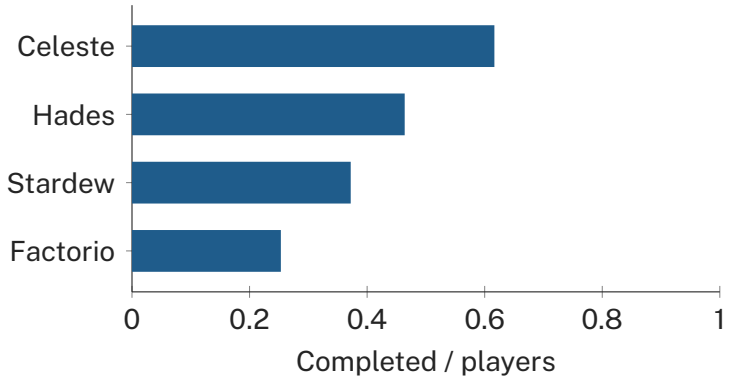


Figure 9. The four eligible game completion rates on a zero-to-one scale.

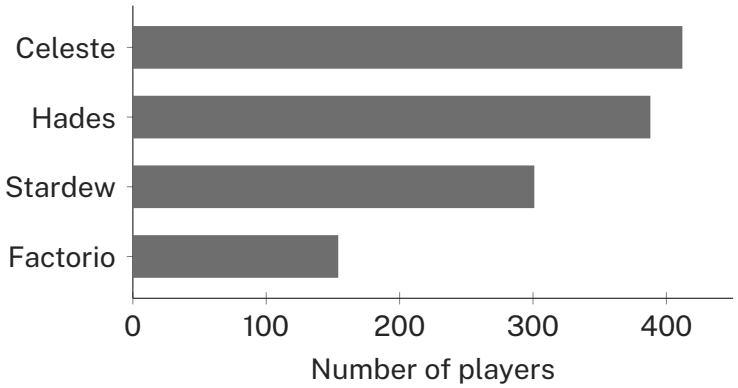


Figure 10. Player counts for the same games, kept in the same order. These are the rate denominators.

To make the second report, change the threshold from 100 to 300. This drops Factorio (154 players). Celeste, Hades and Stardew remain, and their rates do not change; only coverage does. In R, override the YAML default while rendering, writing a separate PDF.

```
Rscript --vanilla -e "rmarkdown::render(
  'report.Rmd', params=list(minplayers=300),
  output_file='report-300.pdf')"
```

In Python, change the parameter cell to `minplayers = 300`, save, and run the command below. The executed source records that value.

```
jupyter nbconvert --execute --to pdf --no-input --output report-300 report.ipynb
```

At a threshold of 1000, no game qualifies. The report should say that no games met the threshold and state the threshold, rather than draw an empty chart that implies evidence.

9 What a reproducible report establishes

What it is for This section separates what a clean render proves from what it does not, and lists what to preserve. Sandve and colleagues give a compact set of rules for the same goal [7].

The rule A clean run shows that the preserved source, data and software produced a particular PDF. It does not show that a sample represents a population, that a cleaning choice is justified, or that a conclusion follows from its chart. Those need separate analytical checks of the data and the method.

Troubleshooting Start with the first informative error in the log, fix that cause, and rerun.

- A missing object or package, or an invalid operation, is an execution error; check the setup and the selected environment.
- A path that cannot be read is a missing input; check the working folder and the relative path.
- A Pandoc error concerns document conversion.
- A XeLaTeX error concerns PDF compilation; read its log.

A working HTML file can isolate a PDF-stage problem. It does not prove the PDF is correct or that every required computation ran.

What to preserve Required computations must not be skipped with `eval=FALSE`, stale caches or unexecuted cells. Relative paths must resolve from the stated folder. Keep, with the final PDF,

- the source, the raw input identity, the parameters and the software versions;
- the units, exclusions, cleaning decisions and analytical limits; and
- the seed of any operation that uses randomness (these examples use none).

Keep source and data in their existing Git history. Commit meaningful increments and submit the full commit SHA. Do not amend, squash, rebase, force-push or backdate submitted work. A checksum identifies a file's bytes; it does not prove who wrote it or when it was submitted. Publish a portfolio version only after the deadline and instructor release.

10 Citing the data source

Cite the official data record inside the report. No bibliography file or citation package is needed; a short Markdown section in the R Markdown source or a notebook Markdown cell is enough. Take the details from the catalogue page and replace the bracketed placeholder with your download date.

```
## Data source

Nova Scotia Department of Energy. *Low Carbon Communities
Grant Recipients* (record p9y5-d99g). Nova Scotia Open
Government Licence. https://data.novascotia.ca/d/p9y5-d99g,
accessed [YYYY-MM-DD].
```

Place the section near the end of the brief and refer to it where the data are first described. Give any further article or report you rely on its own entry in the same form.

11 Summary

- A dynamic document rebuilds text, tables and figures from one source, but only a check of the rendered PDF confirms the result.
- Execution, conversion and TeX compilation are separate stages; the failing stage tells you which log to read.
- Render from a fresh R process or with `--execute`; saved notebook output can come from an earlier run.
- `eval` controls running and `echo` and `include` control showing. They are independent.
- A parameter changes coverage, not the per-row values. Record the value used and report an empty selection plainly.
- A reproducible report shows the source produced the PDF, not that the analysis is right.

12 Ungraded practice

1. Choose R or Python and build the Iris report.
2. Export a fresh PDF and verify its count, mean and plots.
3. Change the game threshold from 100 to 300.
4. Explain the coverage change and distinguish rate from count.

References

[1] J. J. Allaire, C. Teague, C. Scheidegger, Y. Xie, C. Dervieux, and G. Woodhull. *Quarto*. URL: `https://quarto.org/`.
[2] R. A. Fisher. “The use of multiple measurements in taxonomic problems”. In: *Annals of Eugenics* 7.2 (1936), pp. 179–188.
[3] T. Kluyver, B. Ragan-Kelley, F. Pérez, B. Granger, M. Bussonnier, J. Frederic, K. Kelley, J. Hamrick, J. Grout, S. Corlay, P. Ivanov, D. Avila, S. Abdalla, C. Willing, and Jupyter Development Team. “Jupyter Notebooks – a publishing format for reproducible computational workflows”. In: *Positioning and Power in Academic Publishing: Players, Agents and Agendas*. IOS Press, 2016, pp. 87–90.
[4] J. MacFarlane, A. Krewinkel, and J. Rosenthal. *Pandoc: A Universal Document Converter*. URL: `https://pandoc.org/`.
[5] J. F. Pimentel, L. Murta, V. Braganholo, and J. Freire. “A large-scale study about quality and reproducibility of Jupyter notebooks”. In: *Proceedings of the 16th IEEE/ACM International Conference on Mining Software Repositories (MSR)*. IEEE, 2019, pp. 507–517.
[6] R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing. Vienna, Austria, 2026.
[7] G. K. Sandve, A. Nekrutenko, J. Taylor, and E. Hovig. “Ten simple rules for reproducible computational research”. In: *PLOS Computational Biology* 9.10 (2013), e1003285.
[8] J. W. Tukey. *Exploratory Data Analysis*. Reading, MA: Addison-Wesley, 1977.
[9] Y. Xie, J. J. Allaire, and G. Grolemund. *R Markdown: The Definitive Guide*. Chapman and Hall/CRC, 2018.